

Programavimo/projektavimo užduotys

Užduočių reikalavimai ir vertinimo kriterijai

1 užduotis. Užduoties funkcionalumo įgyvendinimas (viso 5 balai)

- 1) Galima naudoti bet kokią objektinę programavimo kalbą (pvz. Java, C#, C++).
- 2) Suprogramuoti užduotį taip, kad programa viektų ir atitiktų pagrindinius reikalavimus (atliktų pagrindines funkcijas, turėtų sąsają su klaviatūra/pelyte, programa turėtų pradžią ir pabaigą).
- 3) Nėra būtina, kad užduotis būtų atvaizduota su išbaigtu grafiniu interfeisu. Vartotojo sąsaja gali būti ir paremta konsole.
- 4) Kodas turėtų būti **objektinis**, t.y. sukurtas objektinis modelis, paskirstytos atsakomybės.
- 5) Prieš atsiskaitymą kodas turi būti įkeltas į github.com arba kitą viešai prieinamą kodo repositoryją ir užpildyti šią formą: <https://forms.gle/fbueopVjAtQjb25W8>

 17 Atsiskaitymo terminas: 2025-11-10 (Po to kiekvieną savaitę balas mažinamas po 1 balą.)

2 užduotis. Refaktoringas (viso 5 balai)

Programa sukurta I užduotyje, turi būti išrefaktorinta taip, kad atitiktų šiuos reikalavimus:

1. OO koncepcijos (1 balas)

- Panaudotos visos 4 pagrindinės OOP koncepcijos praktikoje:

- Inheritance (paveldėjimas): bent viena klasė paveldi iš kitos klasės ir perima arba papildo elgesį.
- Encapsulation (inkapsuliacija): duomenys slepiami privačiuose laukuose, prieiga vykdoma per metodus/getterius/setterius.
- Polymorphism (polimorfizmas): realiai panaudota situacija, kai keli skirtingi objektai gali būti naudojami per bendrą tipą (pvz. sąraše laikomi skirtingų tipų priešai, bet visi kviečiami tuo pačiu metodu update()).
- Abstraction (abstrakcija): panaudotas interfeisas arba abstrakti klasė, kuri apibrėžia bendrą elgesį, o konkrečios klasės jį realizuoja (pvz. interfeisas Enemy su metodu attack(), kurį kitaip įgyvendina skirtingi priešų tipai).

2. Švarus kodas ir DRY principas (2 balai)

- Klasės ≤ 200 eilučių, metodai ≤ 30 eilučių.
- Nėra pasikartojančio kodo (DRY principas).
- Naudojamos konstantos, o ne „magic numbers“.
- Nedidelis kodo sudėtingumas:
 - Vengiama įdėtų ciklų su keliais if/else.
 - Jei logika sudėtinga – ji padalinta į mažesnius metodus/klases.

3. Design pattern'ai (1 balas)

- Panaudotas bent vienas creational pattern (pvz. Factory Method, Abstract Factory, Builder, Prototype).
- Singleton pattern naudoti negalima.
- Panaudotas bent vienas behavioural pattern (pvz. Strategy, Observer, State, Command, Template Method, Mediator).

4. Unit testai (1 balas)

- Sukurta 5–10 unit testų.
- Testai tikrina svarbiausią žaidimo logiką:
 - Objektų būsenų pasikeitimus (pvz. gyvybės sumažėjimą po atakos).
 - Skaičiavimus (pvz. taškų ar balanso pokyčius).
 - Sąveikas tarp objektų (pvz. ar durys atrakintos turint raktą).
- Testai turi būti paleidžiami su atitinkamomis priemonėmis (pvz. JUnit – Java, NUnit – C ir pan.).
- Visi testai turi būti vykdomi be klaidų.



Atsiskaitymo terminas: 2025-12-15

Pastaba: Atsiskaitymai vykdomi gyvai pratybų metu apsiginant darbą. Rekomenduojama išmokti programavimo aplinkos (IDE) pagrindinius shortcut'us, jog būtų galima lengvai naviguoti kode, jį demonstruojant.

Užduočių aprašymai

1. Rūšiotojas

Žaidėjas valdo konvejerio sklendes ir turi nukreipti įvairių tipų dėžes į teisingus kontenerius.

Reikalavimai:

- - Konvejeris nuolat generuoja atsitiktinio tipo dėžes (pvz. A, B, C).
- - Žaidėjas klavišais (1, 2, 3) perjungia sklendes.
- - Dėžė patenka į kontainerį pagal sklendės padėtį.
- - Teisingas nukreipimas prideda taškų, klaida atima gyvybę.
- - Žaidimas baigiasi, kai baigiasi gyvybės arba pasiekiamas taškų limitas.
- - Pabaigoje rodoma rezultatų santrauka.

Papildiniai: Kombo daugikliai, „klaidų serijos“ baudos, lygių progresija.

2. Šviesoforų meistras

Žaidėjas valdo šviesoforus sankryžoje ir turi užtikrinti sklandų eismą be avarijų.

Reikalavimai:

- - Sankryžoje stovi šviesoforai, keičiasi spalvos ciklu.
- - Žaidėjas klavišais keičia šviesoforo fazes (ŽALIA–GELTONA–RAUDONA).
- - Automobiliai atsiranda atsitiktinai ir juda per sankryžą.
- - Jei du automobiliai susiduria, žaidėjas pralaimi.
- - Jei sankryžoje susidaro per ilga spūstis, žaidimas baigiasi.
- - Laimėjimas, jei pavyksta išlaikyti eismą tam tikrą laiką be avarijų.

Papildiniai: Skirtingi sankryžų šablonai, oro sąlygos, avariniai prioritetai.

3. Kosminis kurjeris

Žaidėjas skraidina krovinį tarp planetų. Kiekvienas skrydis kainuoja kurą, o kelionės metu gali nutikti įvairūs pavojingi įvykiai. Reikia pasirinkti saugiausią ir efektyviausią maršrutą.

Reikalavimai:

- - Žemėlapis sudarytas iš planetų (mazgų) ir tarp jų esančių maršrutų (briaunų).
- - Kiekvienas maršrutas turi kuro kainą ir rizikos lygį.
- - Žaidėjas renkasi, į kurią planetą skristi toliau.
- - Atsitiktiniai įvykiai: piratų užpuolimas, kosminė audra, kuro nuotėkis, navigacijos klaida.
- - Žaidėjas turi ribotą kuro kiekį; kuro nepakanka – pralaimėjimas.
- - Krovinys gali būti prarastas dėl įvykio – pralaimėjimas.
- - Laimėjimas, jei krovinys pristatomas į paskirties planetą.

Papildiniai: Draudimas krovinio rizikai, laivo modifikacijos.

4. Požemių robotas

Žaidėjas valdo robotą, kuris klaidžioja po požemius su ribotu matomumu, renka raktus ir ieško išėjimo.

Reikalavimai:

- - Žaidimo lenta – 2D tinklelis su sienomis, raktais, durimis ir spąstais.
- - Žaidėjas valdo robotą strėlytėmis.
- - Matoma tik dalis lauko aplink robotą, likusi dalis paslėpta („fog of war“).
- - Reikia rinkti raktus, atrakinti duris, vengti spąstų.
- - Tikslas – pasiekti išėjimą.
- - Žaidimas baigiasi, jei robotas žūsta spąstuose arba pasiekia išėjimą.

Papildiniai: Energijos limitas, įvairūs robotų moduliai.

5. Burtininko dvikova

Du burtininkai kovoja paeiliui, naudodami burtus ir taktiką, kol vienas iš jų pralaimi.

Reikalavimai:

- - Du burtininkai kovoja paeiliui.
- - Kiekvienas turi gyvybes, maną ir burtų rinkinį.

- - Burtai gali daryti žalą, gydyti, suteikti apsaugą ar sunaudoti maną.
- - Žaidėjas renkasi burto kortą, AI atsako savo veiksmu.
- - Kova tęsiasi, kol vieno burtininko gyvybės ≤ 0 .
- - Rezultate rodoma, kas laimėjo.

Papildiniai: Status efektai (nuodai, nutildymas), kombo sinergijos.

6. Ugnikalnio evakuacija

Miestelyje prasideda ugnikalnio išsiveržimas. Gyventojai turi evakuotis į saugias zonas, kol lavos banga jų nepasiekė. Žaidėjas gali ribotai paveikti aplinką, kad padėtų evakuacijai.

Reikalavimai:

- - Žemėlapis sudarytas iš kelių, namų ir saugių zonų.
- - Gyventojai pradeda namuose ir automatiškai juda trumpiausiu keliu į saugią zoną.
- - Lava progresyviai plečiasi per žemėlapi.
- - Žaidėjas turi ribotą skaičių veiksmų: gali pastatyti barikadą arba atverti naują kelią.
- - Jei lava užklumpa gyventoją – jis žūsta.
- - Žaidimo tikslas – išgelbėti kuo daugiau gyventojų.
- - Žaidimas baigiasi, kai visi gyventojai pasiekia saugią zoną arba visi žūsta.

Papildiniai: Skirtingi agentų tipai (lėti/greiti), evakuacijos prioritetai.

7. Biržos mini-simulatorius

Žaidėjas prekiauja vienu aktyvu, kurio kaina nuolat kinta, bandydamas gauti kuo daugiau pelno iki laiko pabaigos.

Reikalavimai:

- - Vienas aktyvas, kurio kaina kinta atsitiktinai.
- - Žaidėjas turi pradinį pinigų balansą.
- - Galimi veiksmai: BUY, SELL, HOLD.
- - Atliekant pirkimą ar pardavimą atnaujinamas portfelis ir balansas.
- - Tikslas – turėti kuo daugiau pelno iki žaidimo pabaigos.
- - Žaidimas baigiasi, kai baigiasi laikas arba bankrotas.

Papildiniai: Paprasti indikatoriai (SMA), mokesčiai, skirtingi modeliai.

8. Sodo gynyba

Žaidėjas turi apsaugoti daržą nuo kenkėjų bangų, statydamas bokštelius, kurie šaudo į priešus.

Reikalavimai:

- - Yra takas, kuriuo juda kenkėjai bangomis.
- - Žaidėjas turi ribotą biudžetą bokštams statyti.
- - Bokštai turi kainą, atakos nuotolį ir žalą.
- - Bokštai automatiškai šaudo į artimiausius kenkėjus savo zonoje.
- - Jei per daug kenkėjų pasiekia daržą, žaidimas baigiasi.
- - Laimėjimas, jei pavyksta atlaikyti visas bangas.

Papildiniai: Daugiatakliai, bokštų patobulinimai, „užšalimo“ efektas.

9. Lobių sala

Žaidėjas tyrinėja salą ieškodamas paslėpto lobio. Kelionės metu tenka rinktis maršrutus, kurie gali būti pavojingi.

Reikalavimai:

- - Sala padalinta į sektorius (tinklelis arba grafas).
- - Kai kuriuose sektoriuose yra spąstų, sargybinių ar kliūčių.
- - Žaidėjas turi ribotą gyvybių skaičių ir atsargų kiekį.
- - Judant į naują sektorių gali įvykti atsitiktinis įvykis.
- - Tikslas – rasti lobį ir grįžti į starto tašką gyvam.

Papildiniai: Galimybė susirinkti komandos narius, papildomi antriniai lobiai.

10. Miesto meras

Žaidėjas valdo mažą miestą, priimdamas sprendimus dėl resursų paskirstymo ir infrastruktūros statybos.

Reikalavimai:

- - Yra gyventojų skaičius, biudžetas, miesto rodikliai (pvz. laimė, saugumas, aplinka).
- - Žaidėjas kiekviename ture priima sprendimus: statyti, taisyti, didinti mokesčius, mažinti išlaidas.
- - Sprendimai turi teigiamų ir neigiamų pasekmių.
- - Jei biudžetas nukrenta iki nulio arba gyventojų laimė tampa labai žema – pralaimėjimas.
- - Jei miestas išsilaiko nustatytą skaičių turų – laimėjimas.

Papildiniai: Atsitiktiniai įvykiai (gaisras, protestas), kelių miestų valdymas.

Papildomos pastabos!

1) Tikslas nėra sukurti pilnai veikiančią žaidimą. Tikslas yra išmokti rašyti švarų kodą ir jį refaktorinti. Jei funkcionalumo reikalavimai ir užduoties aprašymai nėra iki galo aiškūs, juos galima interpretuoti savaip, pakeisti kitaip arba papildyti savo sugalvotais reikalavimais..

2) Atsiskaitant, kodą reikės apsiginti pratybų metu. Kodas kopijuotas iš interneto, sukurtas draugo, ar padarytas pagal youtube tutorial'ą nebus užskaitytas. AI generuotą kodą privaloma mokėti paaiškinti.

3) Atsiskaičius abi užduotis iki 2025-11-10 vienu kartu*, skiriami papildomos 1 balas.
(Galioja tik jei antra užduotis atlikta 100% pagal reikalavimus)

4) Reikia surinkti 5 iš 10 balus, norint laikyti egzaminą.

5) Užduoties praplėtimas su advanced funkcionalumu (+2 papildomai balai)

- Iš anksto suderinti nebūtina, bet pasikonsultuoti rekomenduojama
- Pvz. web based su backend ar sudėtingesnis funkcionalumas (online multiplayer, ar pan)
- Galimi susitikimai apsitarimui, code review'ai pagal poreikį

Puiki terpė kompetencijos pasikėlimui ir pasiruošimas diplominiam darbui